Stage 3: Nanopublication-Based Knowledge Graph I

Stage 3 is the methodological core of this work: it transforms unstructured abstracts into a structured, RDF-compatible knowledge graph of scientific evidence. The stage encompasses four tightly coupled operations: LLM-based assertion extraction, nanopublication packaging, knowledge graph construction, and citation-weighted hypothesis scoring.

Stage 3: Nanopublication-Based Knowledge Graph II

LLM-Based Assertion Extraction

We extract assertions by prompting a locally hosted LLM (Ollama [ollama2024]) to assess each paper's abstract against eight standard hypotheses. The model receives a structured prompt containing the paper title, abstract, and hypothesis definitions, and returns a JSON array where each element specifies a hypothesis ID, direction (supports, contradicts, neutral, or irrelevant), a confidence score $c \in [0, 1]$, and a reasoning string. Assertions marked "irrelevant" are discarded; confidence values are clamped to $[0, 1]$; and responses are validated against the known hypothesis ID set. Papers lacking abstracts are skipped. Detailed prompt engineering, error taxonomy, and validation methodology are documented in the extraction pipeline section.

Stage 3: Nanopublication-Based Knowledge Graph III

Nanopublication Schema and RDF Structure

Each assertion is encoded as a **nanopublication** [groth2010anatomy, kuhn2016decentralized]—a minimal, self-contained, machine-readable unit of scientific evidence. Formally, each nanopublication is a tuple (p, h, d, c) where p is the paper identifier, h the hypothesis identifier, $d \in \{\text{supports}, \text{contradicts}, \text{neutral}\}$ the direction, and c the confidence. Provenance metadata records the LLM model, UTC timestamp, and paper identifier.

The pipeline serializes nanopublications in two complementary formats:

1. **JSON Lines** (one JSON object per line) for efficient incremental checkpointing. Assertions are saved at configurable intervals (default: every 50 papers), enabling the pipeline to resume from where it left off after interruption without re-processing already-analyzed papers. Deduplication uses the composite key $(paper\_id, hypothesis\_id)$; re-runs